Information Retrieval 8

Cross-Encoder Reranking

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

monoBERT = A Cross-encoder that inputs `[CLS] q [SEP] d [SEP]` and estimates relevance from the [CLS] representation. Achieved 2x improvement over BM25 on MS MARCO

Why BERT works = Self-Attention simultaneously captures exact matching + semantic matching + structural cues

Birch = Splits into sentences and transfers zero-shot. Estimates document relevance using the highest-scoring sentence

BERT-MaxP = Splits into passages and aggregates by maximum score. Effective even with noisy labels

This time we learn advanced forms of the Cross-encoder: **monoT5** , **duoT5** , and **LLM-based reranking**

Understanding the evolution of Cross-encoder reranking along three axes: Pointwise -> Pairwise -> Listwise, BERT -> T5 -> LLM, and the tradeoff between efficiency and effectiveness

---

First Half: Reranking with T5

- Pointwise -> Pairwise -> Listwise
- monoT5: Seq2Seq Approach
- duoT5: Pairwise Comparison
- Multi-stage Ranking Pipeline

Second Half: Reranking with LLMs

- Efficiency via Knowledge Distillation
- RankGPT: LLM Listwise Reranking
- RankZephyr / RankLLaMA
- Summary of Efficiency and Effectiveness

Three Approaches to Reranking

Pointwise, Pairwise, Listwise

Pointwise (one document at a time):

Scores each document **independently**

$s(q, d_i) \rightarrow \mathbb{R}$

Examples: monoBERT, monoT5

Pairwise (comparing two documents):

Determines which of a document pair (d_i, d_j) is more relevant

$P(d_i \succ d_j \mid q) \rightarrow [0, 1]$

Example: duoT5

Listwise (reordering the entire list):

Takes the entire candidate list as input and directly outputs a permutation

$\{d_1, ..., d_n\} \rightarrow \pi(d_1, ..., d_n)$

Example: RankGPT

Comparison of Approaches

Pointwise

Complexity: O(n) inference calls
Accuracy: Does not consider relative relationships between documents
Simplest and fastest

Pairwise

Complexity: O(n^2) or O(n log n) inference calls
Accuracy: Determines rankings through relative comparisons
Improves accuracy but increases computational cost

Listwise

Complexity: O(1) inference calls (but large input length)
Accuracy: Considers the context of the entire list
Requires long context length of LLMs

Limitations of monoBERT:

1. **Encoder-only constraint**: Since BERT solves ranking as a classification task, the output is limited to a single relevance score
2. **Pointwise limitation**: Since each document is scored independently, it cannot directly model relative relationships between documents
3. **Fixed architecture**: BERT's pre-training is optimized for classification, making it difficult to leverage ranking-specific structures

Directions for advancement:

Leveraging Seq2Seq Models

Reformulating ranking as a generation task using Encoder-Decoder models like T5
-> monoT5, duoT5

Introducing Pairwise Comparison

Inputting two documents simultaneously and selecting the more relevant one
-> duoT5

Leveraging LLMs

Reordering the entire list using the reasoning capability of large language models
-> RankGPT

monoT5

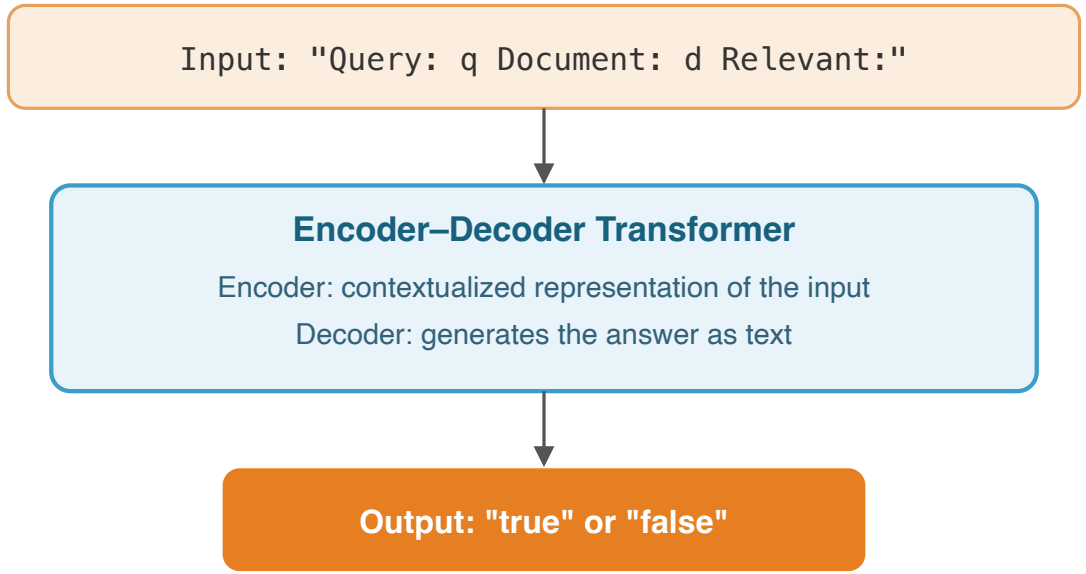
Seq2Seq Relevance Ranking

T5 [Raffel et al., 2020] uniformly treats all NLP tasks as **text generation tasks**:

- Input: Text (task instruction + input data)
- Output: Text (task answer)
- Examples: translation, classification, summarization (see figure)

Differences from BERT:

- BERT: Encoder-only -> Classification from [CLS] representation
- T5: Encoder-Decoder -> Generates answer as text



translation: `"translate English to German: ..."` → German text
classification: `"sst2 sentence: ..."` → "positive" / "negative"
summarization: `"summarize: ..."` → summary text

Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer", JMLR, 2020

monoT5 [Nogueira et al., 2020] = Applying T5 to Pointwise reranking

Input Template:

```
Query: {q} Document: {d} Relevant:
```

Output: `"true"` or `"false"` token

Scoring:

$$s(q,d) = \log P(\texttt{"true"} \mid q,d) - \log P(\texttt{"false"} \mid q,d)$$

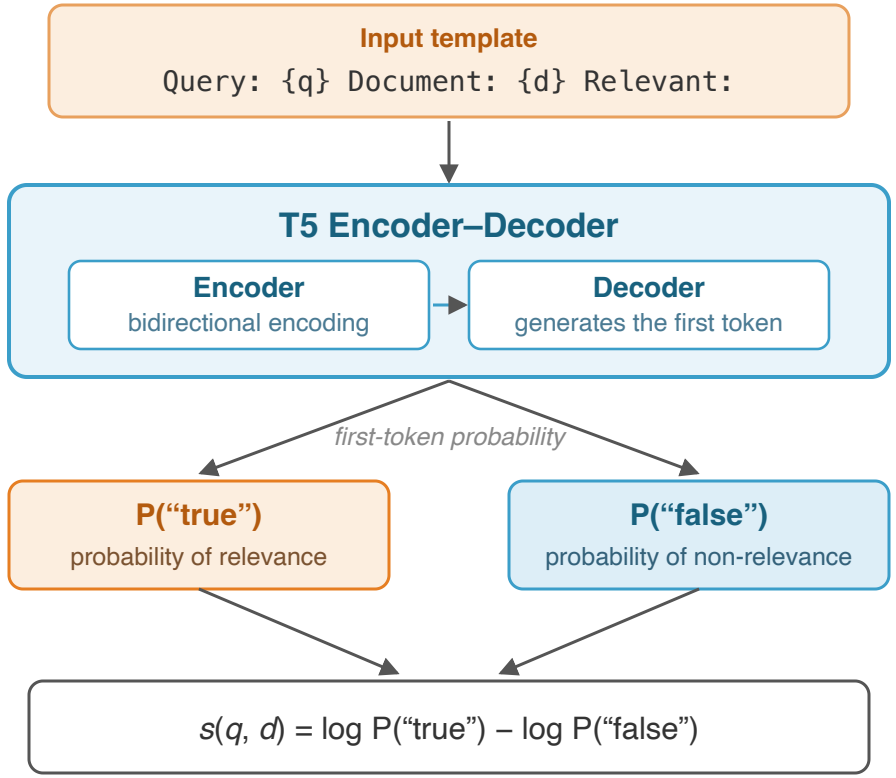
Uses only the first-token generation probability of the Decoder

Training:

Relevant pair (q, d^+) -> Target `"true"`

Non-relevant pair (q, d^-) -> Target `"false"`

Trained with standard Seq2Seq cross-entropy loss



The key insight is transforming ranking into **"answering relevance as text"**. Instead of relying on BERT's classification head, it leverages T5's generation capability

Nogueira et al., "Document Ranking with a Pretrained Sequence-to-Sequence Model", Findings of EMNLP, 2020

Comparison on MS MARCO passage ranking:

Method	Parameters	MRR@10
BM25	—	0.184
monoBERT-Large	340M	0.372
monoT5-Base	220M	0.381
monoT5-Large	770M	0.393
monoT5-3B	3B	0.398

monoT5-Large achieves **comparable or better** performance than monoBERT-Large

monoT5-3B achieves a significant improvement with **MRR@10 = 0.398**

Key findings:

Scaling Effect

In this comparison setting, T5 shows performance improvements as model size increases. The 3B parameter model demonstrated particularly high performance

Advantage of the Generation Framework

Natural language output of "true"/"false" is more flexible than a classification head. It directly leverages linguistic knowledge from pre-training

Practical Aspects

Inference is Pointwise with the same O(n) as monoBERT, but slower due to the larger model size

Nogueira et al., "Document Ranking with a Pretrained Sequence-to-Sequence Model", Findings of EMNLP, 2020

duoT5

Pairwise Relevance Comparison

duoT5 [Pradeep et al., 2021] = Applying T5 to **Pairwise** reranking

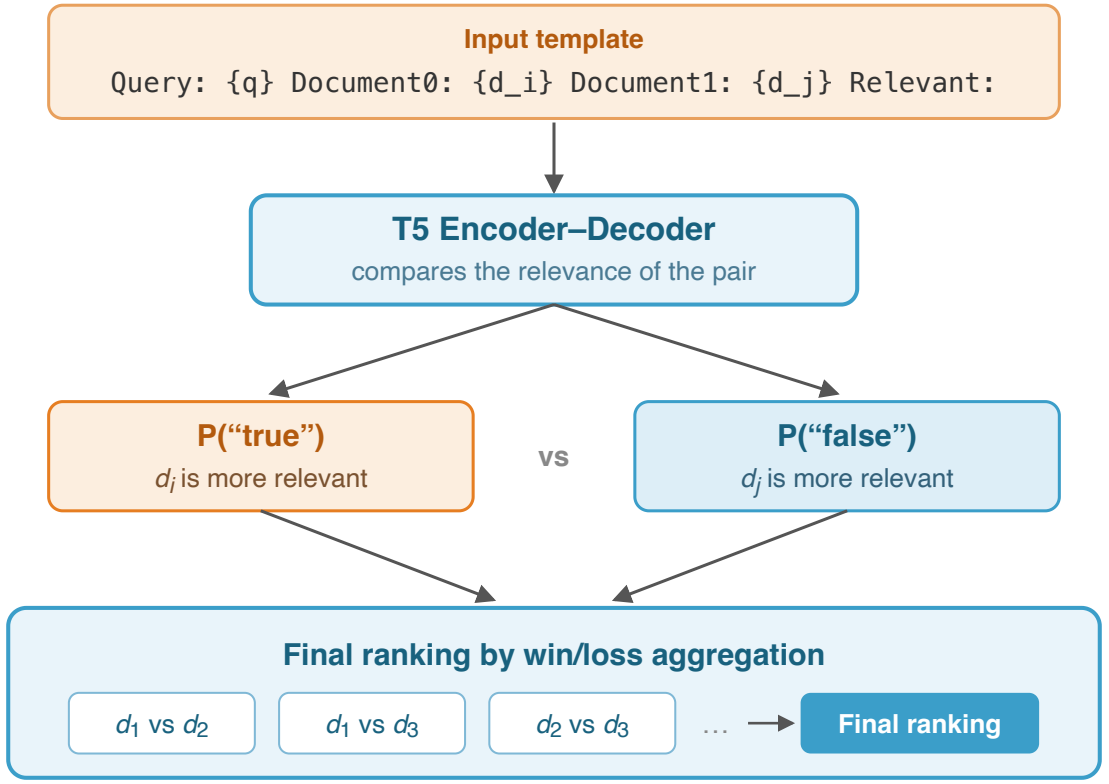
Input Template:

```
Query: {q}
Document0: {d_i}
Document1: {d_j}
Relevant:
```

Output: `"true"` (d_i > d_j) or `"false"` (d_j > d_i)

Score Aggregation: count each document's pairwise "wins"

$$s_{\text{duo}}(q, d_i) = \sum_{j \neq i} 1[P(d_i \succ d_j \mid q) > 0.5]$$



all pairs: O(n^2) inferences / sort-based: O(n log n)

In practice duoT5 is applied only to the top-k candidates (e.g., k = 50) after monoT5

Pradeep et al., "The Expando-Mono-Duo Design Pattern for Text Ranking with Pretrained Sequence-to-Sequence Models", arXiv:2101.05667, 2021

Results on MS MARCO passage ranking:

Method	MRR@10	Inferences/query
BM25	0.187	—
monoT5-3B	0.398	1,000
mono + duoT5-3B (Top 50)	0.404	1,000 + 1,225
mono + duoT5-3B (Top 10)	0.402	1,000 + 45

duoT5 further improves monoT5, but the **improvement margin is small**

All-pairs comparison over Top 50: $\binom{50}{2} = 1{,}225$ inferences

Advantages and challenges of Pairwise:

Advantages

By directly comparing two documents, it can capture subtle differences in relevance. Particularly effective for reordering the top results

Challenge: Computational Cost

All-pairs comparison is $O(n^2)$. Even with n=50, 1,225 inferences are required. Can be reduced to O(n log n) with bubble sort or heap sort, but still significantly more than Pointwise's n calls

duoT5 is a method that "pays cost for accuracy." In practice, it is predicated on a **multi-stage configuration** where monoT5 first narrows down to the top k candidates

Pradeep et al., "The Expando-Mono-Duo Design Pattern for Text Ranking with Pretrained Sequence-to-Sequence Models", arXiv:2101.05667, 2021

Pradeep et al. [2021] proposed a three-stage pipeline:

Stage 1: Expando (Document Expansion)

Augment documents with queries via doc2query, then perform initial retrieval with BM25

-> Retrieve top 1,000 candidates

Stage 2: Mono (Pointwise)

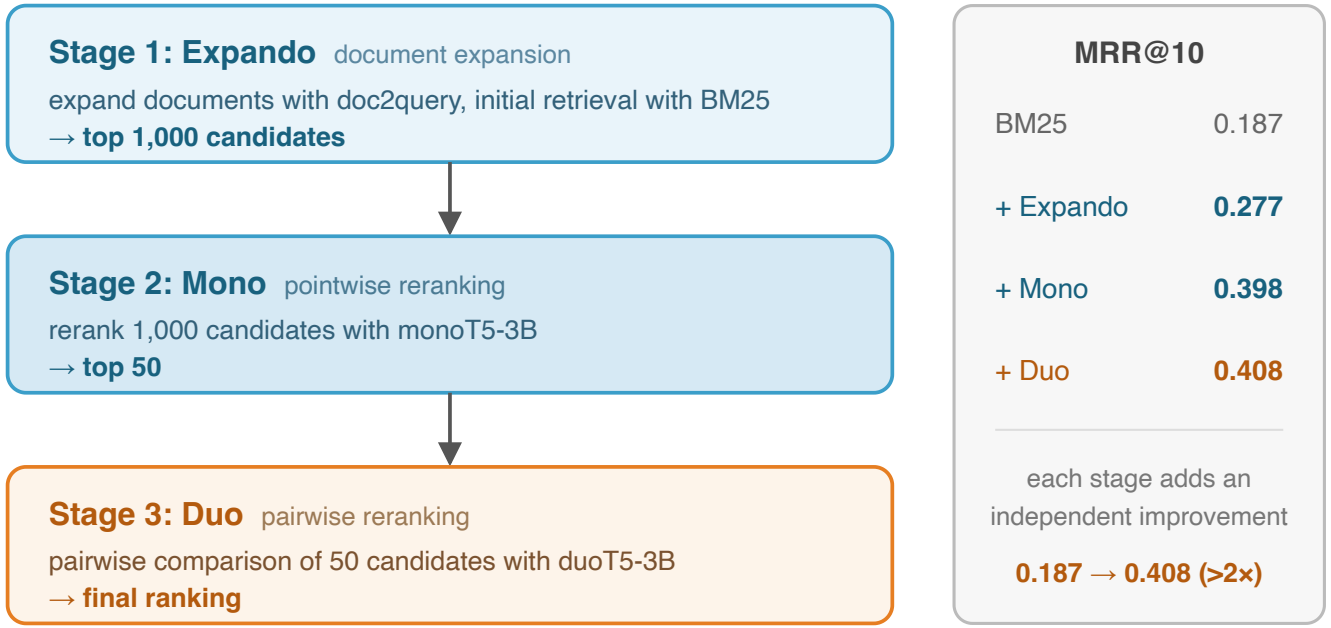
Rerank 1,000 candidates with monoT5-3B

-> Narrow down to top 50

Stage 3: Duo (Pairwise)

Perform pairwise comparison of 50 candidates with duoT5-3B

-> Output the final ranking



Expansion improves Recall offline; Mono prunes cheaply; Duo refines the very top precisely

duoT5 over top 50: C(50, 2) = 1,225 inferences — affordable only after Mono's pruning

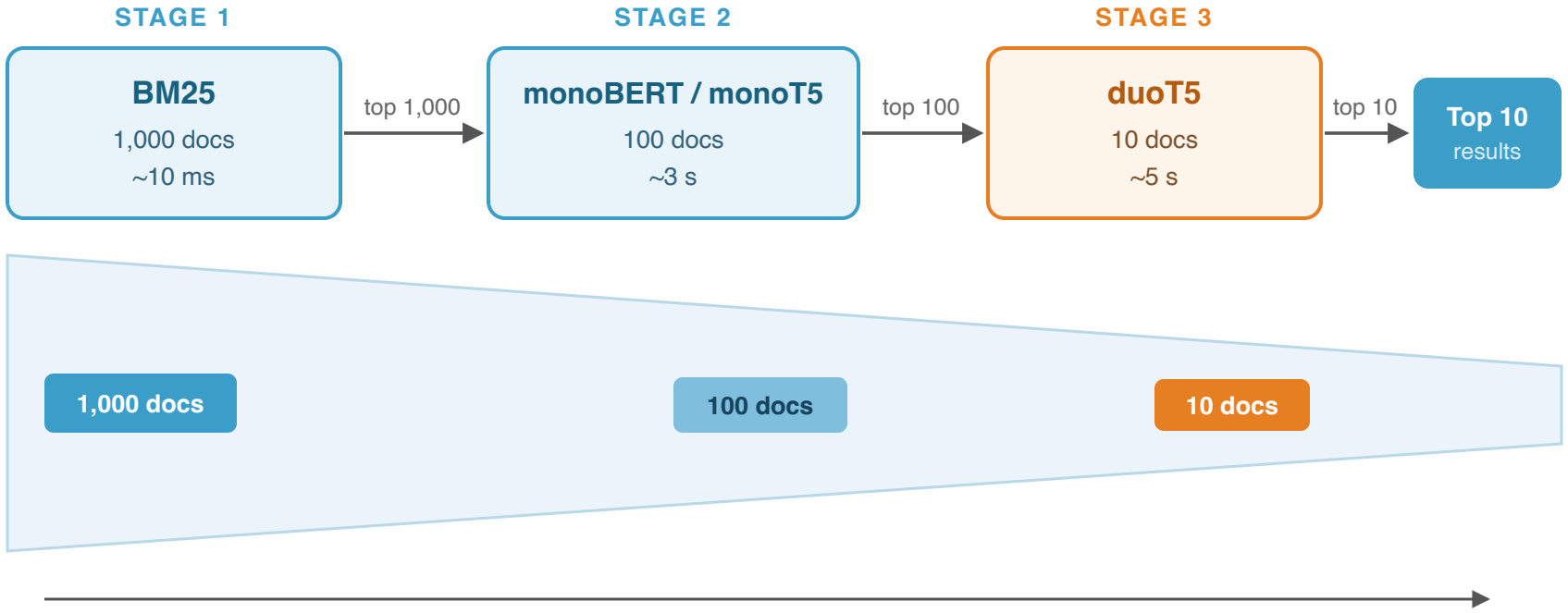
Pradeep et al., "The Expando-Mono-Duo Design Pattern for Text Ranking with Pretrained Sequence-to-Sequence Models", arXiv:2101.05667, 2021

Multi-Stage Ranking

Multi-Stage Ranking Pipelines

Multi-stage ranking = Applying multiple rerankers progressively, narrowing down candidates at each stage — from fast BM25 over the full corpus to an expensive pairwise reranker (see figure)

1. **Accuracy-speed tradeoff**: Later stages use more accurate but slower models
2. **Progressive candidate reduction**: Candidates are significantly reduced at each stage (telescoping)
3. **Additive improvement**: Each stage independently improves accuracy



Lin et al., "Pretrained Transformers for Text Ranking: BERT and Beyond", 2021, § 3.4

Knowledge Distillation

Knowledge Distillation for Efficient Reranking

Knowledge Distillation [Hinton et al., 2015] = A technique for transferring knowledge from a large "teacher model" to a small "student model"

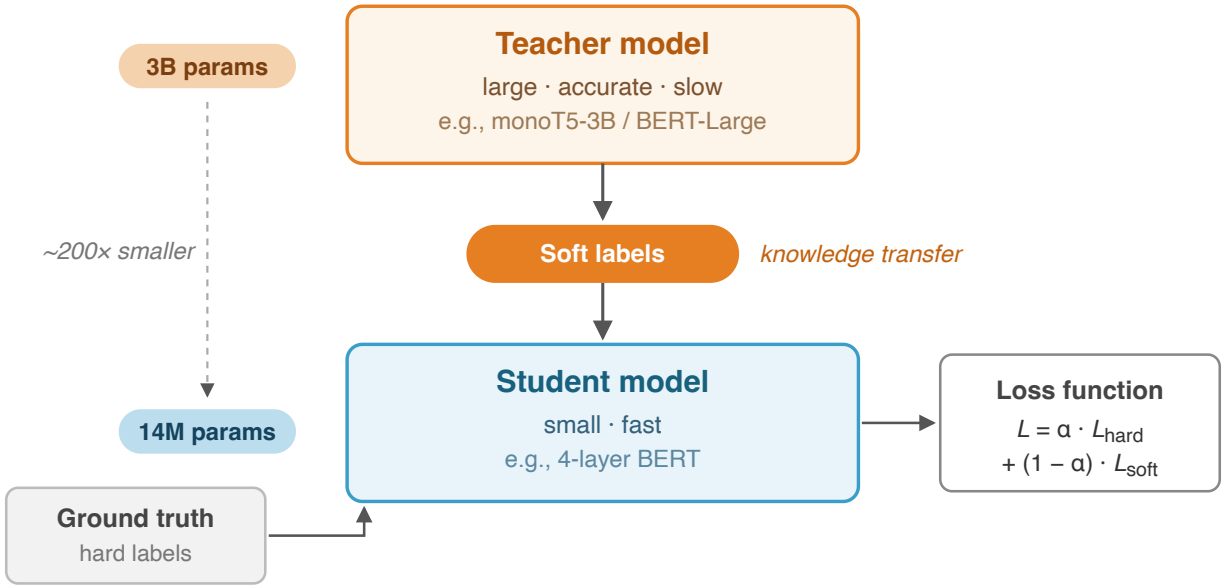
Basic Framework:

1. **Teacher Model**: Highly accurate but large and slow (e.g., monoT5-3B)
2. **Student Model**: Small and fast but less accurate (e.g., 4-layer BERT-Base)
3. **Distillation**: Train the student to mimic the teacher's outputs (soft labels)

Loss Function:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{hard}} + (1 - \alpha) \cdot \mathcal{L}_{\text{soft}}$$

$\mathcal{L}_{\text{hard}}$: ground-truth loss; $\mathcal{L}_{\text{soft}}$: KL divergence with the teacher



A 4-layer student nearly matches the 12-layer teacher at ~3× faster inference (Gao et al., 2020)

Hinton et al., "Distilling the Knowledge in a Neural Network", NIPS Workshop, 2015

Three distillation methods [Gao et al., 2020]:

(1) Ranker Distillation:

Mimics the **ranking order** of the teacher's scores

The student aims to rank in the same order as the teacher

(2) LM Distillation + Fine-tuning:

First initialize with TinyBERT's LM distillation

Then fine-tune on the ranking task

(3) LM + Ranker Distillation:

Combines (1) and (2)

Initialize with LM distillation -> Final training with ranker distillation

Comparison of Distillation Methods (MS MARCO)

Method	Layers	MRR@10	Latency
monoBERT-Base	12	0.347	1x
Ranker Distill.	4	0.332	~3x faster
LM + FT	4	0.308	~3x faster
LM + Ranker Distill.	4	0.339	~3x faster

Based on results from Gao et al. (2020)

A 4-layer model nearly matches the performance of a 12-layer model. Latency is approximately 3x faster. The combination of LM distillation and ranker distillation is most effective

Gao et al., "Understanding BERT Rankers Under Distillation", ICTIR, 2020

Reranking with LLMs

Reranking with Large Language Models

RankGPT [Sun et al., 2023] = Presents a list of candidate documents to an LLM and has it generate a **permutation sorted by relevance**

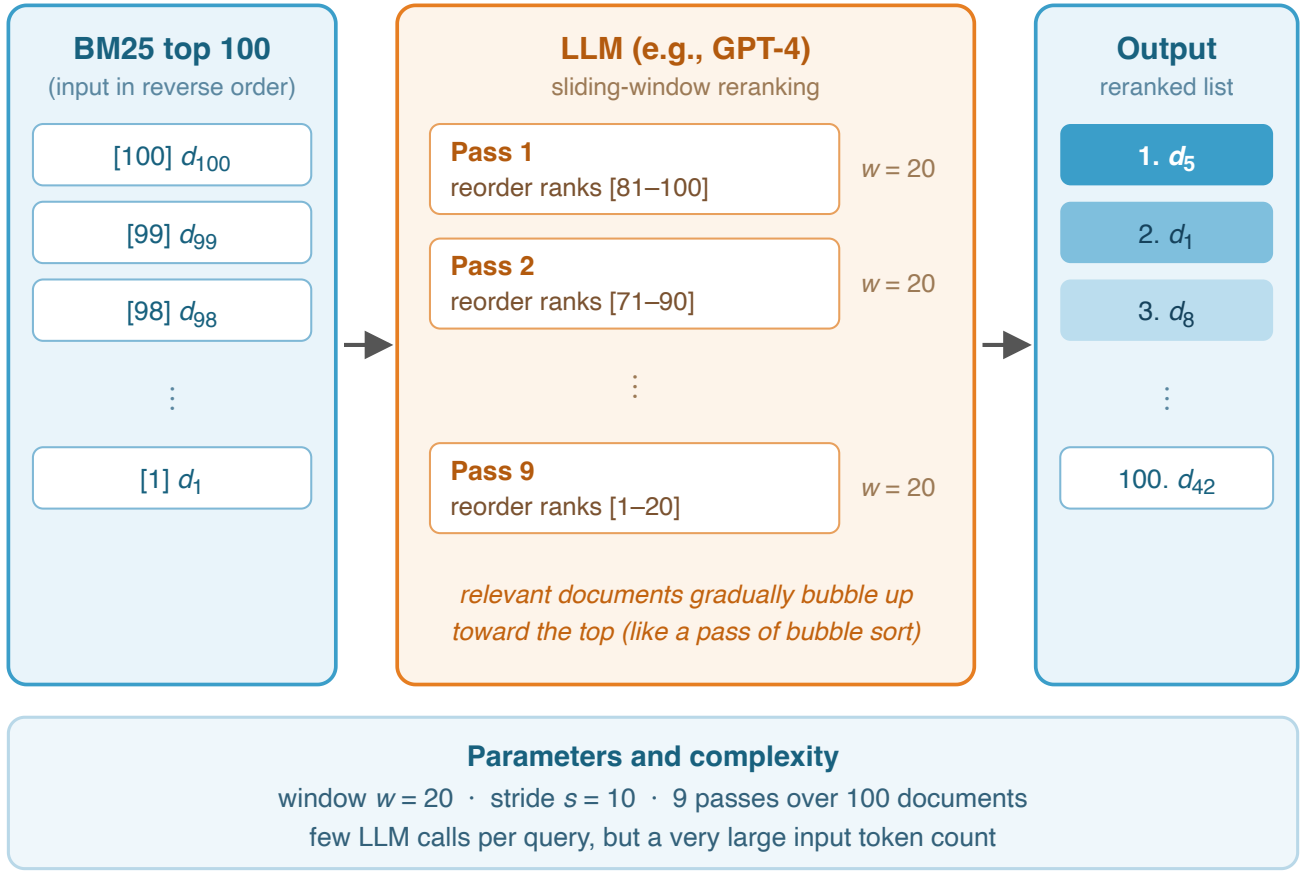
Prompt Structure:

```
I will provide a query and N passages.

Query: {q}
[1] {d_1}
[2] {d_2}
...
[N] {d_N}

Rank the passages by relevance to
the query. Output identifiers from
most to least relevant.
```

Output: `[3] > [1] > [5] > [2] > [4]`



Sun et al., "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents", EMNLP, 2023

Results on TREC DL 2019/2020 (passage):

Method	nDCG@10 (DL19)	nDCG@10 (DL20)
BM25	0.506	0.480
monoBERT	0.705	0.672
monoT5-3B	0.732	0.707
RankGPT (GPT-3.5)	0.659	0.622
RankGPT (GPT-4)	0.755	0.704

Analysis:

GPT-4 achieves performance **comparable to or better than** supervised monoT5-3B in a Zero-shot setting

GPT-3.5 does not match monoBERT

Model scale and reasoning ability significantly affect performance

Important implication: GPT-4 level LLMs can match supervised ranking models without fine-tuning. However, API cost and latency remain major practical challenges

Sun et al., "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents", EMNLP, 2023

Method	Mode	Model Size	FT	Inferences	DL19	DL20
monoBERT-Large	Pointwise	340M	Yes	n	0.705	0.672
monoT5-3B	Pointwise	3B	Yes	n	0.732	0.707
RankGPT (GPT-3.5)	Listwise	~175B	No	n/w passes	0.659	0.622
RankGPT (GPT-4)	Listwise	~1.8T	No	n/w passes	0.755	0.704
RankZephyr	Listwise	7B	Yes	n/w passes	0.752	0.716
RankLLaMA	Pointwise	7B	Yes	n	0.764	0.729

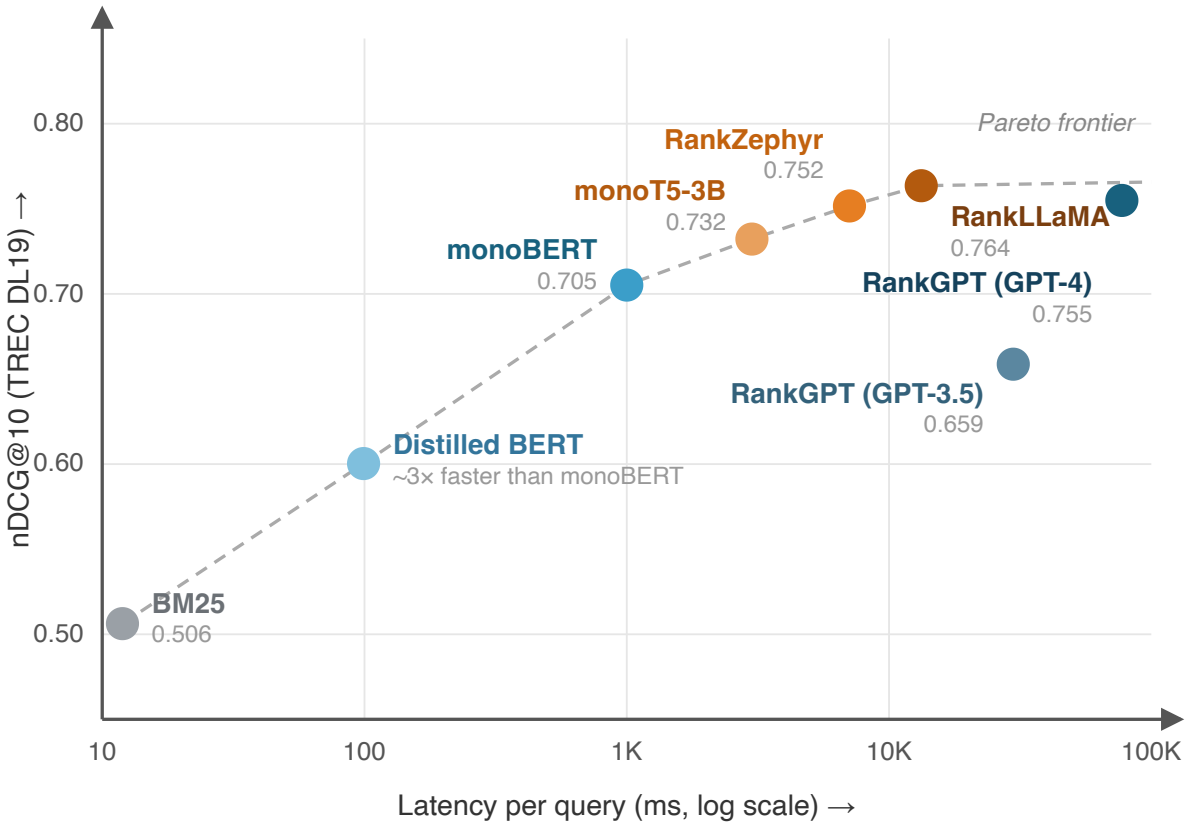
The return of Pointwise:

- RankLLaMA matches Listwise with Pointwise
- Inference cost is O(n), more efficient than Listwise
- When fine-tuning is possible, Pointwise is advantageous

The value of Listwise:

- Effective when fine-tuning is difficult
- High Zero-shot performance (GPT-4)
- Knowledge distillation is also possible via RankZephyr

nDCG@10 values are reported values from each paper. Note that experimental conditions are not fully unified



Conceptual placement — nDCG@10 values are as reported on TREC DL19; latencies are rough estimates and experimental conditions differ across papers

Selection guidelines:

Requirement	Recommended Method
Highest accuracy (cost no object)	RankGPT (GPT-4)
High accuracy + local execution	RankLLaMA / RankZephyr
Balance of accuracy and speed	monoT5-3B
Low latency priority	Distilled BERT
Large-scale service	BM25 + distilled model

In real systems, the optimal method varies depending on the use case, budget, and latency requirements. Combining multiple methods in a multi-stage pipeline is common practice

Design choices for Cross-encoder reranking:

Design Axis	Options	Tradeoff
Approach	Pointwise / Pairwise / Listwise	Accuracy vs Computation
Base Model	BERT / T5 / LLM	Accuracy vs Model Size
Fine-tuning	Yes / No (Zero-shot)	Accuracy vs Data Requirements
Multi-stage Config	Number of stages and candidates per stage	Accuracy vs Latency
Efficiency	Distillation / Quantization / Pruning	Speed vs Accuracy Loss

Research frontiers:

- How to leverage the reasoning ability of LLMs
- Pursuing the Pareto optimum of efficiency and accuracy
- Domain adaptation and generalization performance

Standard configuration in practice:

- Stage 1: BM25 or Dense Retrieval (1,000 candidates)
- Stage 2: monoT5 or RankLLaMA (top 50-100 candidates)
- Add Stage 3 as needed

1. **monoT5** = Applies T5 to Pointwise reranking. Scores based on generation probability of "true"/"false". Achieves MRR@10 = 0.398 with the 3B model
2. **duoT5** = Determines relative document order through Pairwise comparison. Improves accuracy but incurs O(n^2) computational cost. The multi-stage pipeline (Expando-Mono-Duo) is practical
3. **Knowledge Distillation** = Transfers knowledge from a large teacher model (monoT5-3B) to a small student model. A 4-layer BERT nearly matches 12-layer performance at approximately 3x faster speed
4. **RankGPT** = A method that has LLMs rerank via Listwise permutation generation. GPT-4 matches supervised models in Zero-shot. Open-sourced through RankZephyr/RankLLaMA

Next Lecture Preview: Query and Document Expansion

doc2query (generating queries from documents to expand the index)

DeepCT (learning contextual term importance)

DeepImpact (integrating expansion and weighting)